MANUAL TÉCNICO

SISTEMA DE GESTIÓN COMERCIAL

---

1. INFORMACIÓN GENERAL DEL PROYECTO

- **Nombre del Sistema:** Sistema de Gestión Comercial
- **Lenguaje de Programación:** Python 3

---

2. ARQUITECTURA Y ESTRUCTURA DEL PROYECTO

El sistema está diseñado bajo una arquitectura modular en capas donde la lógica de la aplicación se encuentra separada según responsabilidades específicas (Gestión de Productos, Inventarios, Ventas, Reportes y Persistencia).

```
None
sistema-gestion-comercial/
├── README.md
├── .gitignore
├── src/
│   ├── main.py
│   ├── productos.py
│   ├── ventas.py
│   ├── inventario.py
│   ├── archivos.py
│   └── reportes.py
└── docs/
    ├── analisis/
    ├── diagramas/
    ├── manual-tecnico.md
    └── manual-usuario.md
```

---

3. DESCRIPCIÓN DE MÓDULOS Y FUNCIONES

3.1 Módulo `archivos.py`

Proporciona la capa de abstracción para la lectura y escritura de información en formato JSON con soporte de codificación UTF-8.

- **`cargar_datos(nombre_archivo: str) -> list`**
    - **Propósito:** Carga datos estructurados desde un archivo JSON local.
    - **Comportamiento:** Si el archivo no existe o ocurre una excepción, retorna una lista vacía (`[]`).
- **`guardar_datos(nombre_archivo: str, datos: list/dict) -> None`**
    - **Propósito:** Serializa y almacena datos en formato JSON formateado (`indent=4`).

3.2 Módulo `productos.py`

Administra el catálogo general de productos de la empresa.

- **`registrar_producto()`**: Solicita y valida código (único), nombre, categoría, precio (> 0), cantidad (≥ 0) y stock mínimo (≥ 0). Persiste en `productos.json`.
- **`listar_productos()`**: Formatea y despliega la lista general de productos registrados.
- **`consultar_producto()`**: Búsqueda secuencial de productos mediante su código único.

3.3 Módulo `inventario.py`

Controla las existencias físicas y el flujo de movimientos (entradas/salidas).

- **`consultar_existencias()`**: Despliega tabla comparativa entre disponible y stock mínimo.
- **`registrar_entrada_inventario()`**: Aumenta el stock existente y crea un registro histórico en `inventario.json`.
- **`registrar_movimiento(codigo, tipo, cantidad, descripcion)`**: Registra eventos auditables en el historial.
- **`verificar_stock_minimo()`**: Filtra e identifica productos en estado crítico.
- **`consultar_historial_movimientos()`**: Lista la trazabilidad cronológica de entradas y salidas.

3.4 Módulo `ventas.py`

Gestiona la ejecución de ventas, carrito de compra temporal, validación de stock y emisión de facturación básica.

- **`registrar_venta()`**:
  - Implementa un ciclo de recolección de productos.
  - Verifica disponibilidad suficiente en stock.
  - Calcula Subtotal, Impuesto (IVA al 12%) y Total general.
  - Descuenta existencias de `productos.json`.
  - Registra el movimiento en `inventario.json`.
  - Almacena la transacción en `ventas.json`.
- **`consultar_historial_ventas()`**: Muestra el desglose de ventas realizadas.

3.5 Módulo `reportes.py`

Genera resúmenes ejecutivos y análisis estadísticos a partir de las fuentes de datos.

- **`reporte_listado_productos()`**: Salida tabular detallada de catálogo.
- **`reporte_stock_minimo()`**: Alertas críticas de reabastecimiento.
- **`reporte_historial_ventas()`**: Bitácora de transacciones comerciales.
- **`reporte_total_ingresos()`**: Cálculo acumulado de ingresos por ventas.
- **`reporte_producto_mas_vendido()`**: Identifica el código de producto con mayor volumen de salida.
- **`reporte_valor_inventario()`**: Determina el capital inmovilizado en mercancía (`$\sum \text{precio} \times \text{cantidad}$`).
- **`reporte_promedio_ventas()`**: Calcula la media de facturación por transacción (`$\frac{\text{Ingresos Totales}}{\text{Número de Ventas}}$`).

3.6 Módulo `main.py`

Punto de entrada principal de la aplicación. Maneja el menú interactivo por consola y el enrutamiento de flujos a submenús interactivos.

---

4. ESQUEMA DE BASE DE DATOS (JSON)

4.1 `productos.json`

```JSON
[
    {
        "codigo": "STRING (UNICO)",
        "nombre": "STRING",
```

```
        "categoria": "STRING",
        "precio": "FLOAT",
        "cantidad": "INTEGER",
        "stock_minimo": "INTEGER",
        "activo": "BOOLEAN"
    }
]
```

4.2 ventas.json

```
JSON
[
    {
        "numero_venta": "STRING (EJ: V-0001)",
        "fecha": "STRING (YYYY-MM-DD HH:MM:SS)",
        "productos": [
            {
                "codigo": "STRING",
                "nombre": "STRING",
                "precio_unitario": "FLOAT",
                "cantidad": "INTEGER",
                "subtotal": "FLOAT"
            }
        ],
        "subtotal": "FLOAT",
        "iva": "FLOAT",
        "total": "FLOAT"
    }
]
```

4.3 inventario.json

```
JSON
[
    {
        "fecha": "STRING (YYYY-MM-DD HH:MM:SS)",
        "codigo_producto": "STRING",
        "tipo_movimiento": "STRING (ENTRADA | SALIDA/VENTA)",
        "cantidad": "INTEGER",
        "descripcion": "STRING"
    }
]
```